Árboles y recursión

Los árboles son una estructura de datos recursiva compuesta por un elemento denominado *raíz* y por árboles asociados, los cuales se denominan *subárboles*. El hecho de definir la estructura de datos en términos de sí misma es lo que genera que se denomine recursiva. Es por eso que resulta indispensable poder comprender los siguientes conceptos de recursividad:

¿Qué es recursión?

Según se define en los fundamentos de programación: "Es una herramienta muy potente y útil para la resolución de problemas"[1].

---

[1] Fundamentos de programación
http://ocw.upm.es/ciencia-de-la-computacion-e-inteligencia-artificial/fundamentos programacion/contenidosteoricos/ocwfundamentosprogramaciontema7.pdf

Estos problemas están basados en su propia definición.

☰ ¿Qué es una estructura recursiva?

Es una estructura de datos en la que los elementos se definen a sí mismos.

☰ ¿Qué es un método recursivo?

Es un método que directa o indirectamente se hace una llamada a sí mismo.

☰ ¿Cuáles son los problemas de la recursión?

Ciclo infinito.

☰ ¿Solución?

Llamadas con instancias diferentes de menor complejidad.

En el siguiente ejemplo, vamos a ver cómo es posible solucionar el problema de calcular la factorial (*n!*) de un número de forma recursiva.

Por definición matemática, el factorial de un número se calcula con la fórmula representada en la siguiente imagen:

Figura 1:

Factorial (n!):

```
Factorial(0) = 1
Factorial(N) = N * Factorial(N-1)
```

Instancias:

```
0! = 1
1! = 1 * (1 - 1)! = 1 * 0! = 1 * 1 = 1
2! = 2 * (2 - 1)! = 2 * 1! = 2 * (1 * 0!) = 2
3! = 3 * (3 - 1)! = 3 * 2! = 3 * (2 * 1!) = 6
…
```

Para resolver el problema de forma iterativa (sin recursión), tenemos que iterar *n* veces y, en cada iteración, restar una unidad y multiplicar el resultado por el valor de la resta, como se muestra en la siguiente figura:

Figura 2:

```
public int factorialI(int n) {
    int resultado = 1;
    if (n > 0) {
        resultado = n;
        while (n > 1) {
            n = n - 1;
            resultado = resultado * n;
        }
    }
    return resultado;
}
```

Dado que la función se define en términos de sí misma, es posible resolver el problema recursivamente, como se muestra en la siguiente figura:

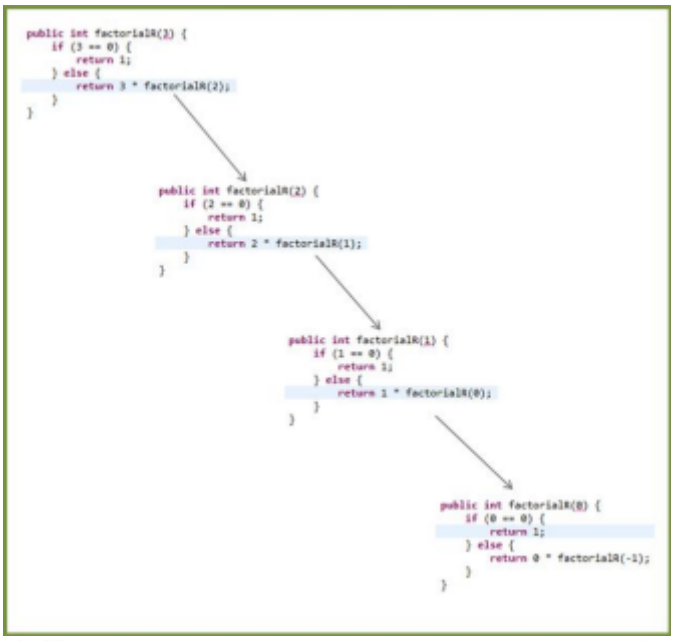
Figura 3:

```
public int factorialR(int n) {
    if (n == 0) {
        return 1;
    } else {
        return n * factorialR(n - 1);
    }
}
```

El algoritmo recursivo es más simple que el algoritmo iterativo, ya que describe de forma similar la fórmula matemática de factorial de *n*.

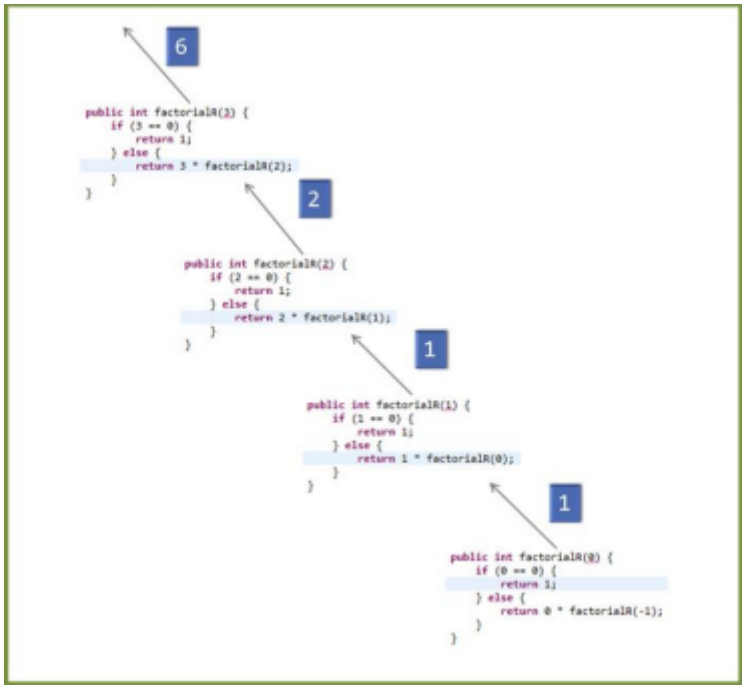
A continuación, vamos a ver cómo funciona el algoritmo en ejecución para ***n* = 3**. En cada llamado al método **factorialR**, se intenta resolver el problema pasando como parámetro un número más simple que el anterior para que el algoritmo lo pueda resolver.

Figura 4:



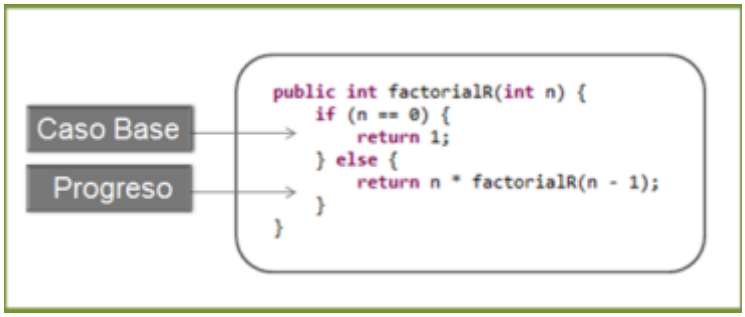
Luego de llamar a **factorialR** con ***n* = 0**, podemos observar que el método retorna **1** y cada llamada anterior comienza a tener los valores correspondientes para poder realizar la multiplicación y retornar el valor de la función factorial.

Figura 5:



Los métodos recursivos se caracterizan por tener un caso base y un progreso. El caso base resuelve el problema sin recursión y el progreso es la llamada recursiva que progresa hacia un caso base. En la siguiente figura, se identifican el caso base y el progreso del algoritmo de factorial visto anteriormente:

Figura 6:



Referencias

Mark Allen Weiss, “Estructuras de Datos en Java”, Adisson Weisley - 2000 - S/D - Bs As

Sznajdleder, P. A. (2012). Algoritmos a fondo con implementación en C y JAVA. Buenos Aires: Alfaomega.

T. Cormen, C. Leiserson, R. Rivest, C.Stein, “Introduction to Algorithms, 2nd Edition.” McGraw Hill. 2002. Sun Press, Core Java Vol. 1 Fundamentals (a proveer por la cátedra) Sun, Java 1.5 API Especification (a proveer por la cátedra) O"Reilly, Eclipse y Eclipse Cookbook (a proveer por la cátedra)